

增強課程 2：Classic User-Exit 實戰——批號自動給號 + Number Range Object

Lecture

這題的起點是一個真實需求：批號要自動依「日期 + 流水號」格式給號，不要用系統預設的批號。這是很典型會用到 Enhancement Framework 的場景——SAP 標準的批號自動給號邏輯是固定的，要改成自訂格式，就得找到官方留的插入點，而不是去改標準程式碼本身。

先講一個重要的系統事實：批號欄位 (**MCHA-CHARG**，Data Element **CHARG_D**，Domain **CHARG**) 長度限制是 **CHAR(10)**——這是本題查證出來的第一個硬限制，直接決定了格式怎麼設計。原本設想的「日期 + 時間 + 流水號 5 碼」(**YYYYMMDD**8碼 + **HHMMSS**6碼 + 5碼 = 19碼) 完全塞不進 10 碼，這門課實際採用的格式是 **YYMMDD** (6 碼) + 流水號 (4碼) = 10碼。

找真正的機制：三代擴充技術疊在同一個地方

用 **sap-adt** MCP 追查 SAP 標準批號自動給號邏輯 (Function Module **VB_NEXT_BATCH_NUMBER**，Function Group **V01Z**，套件 **VB**) 的原始碼，發現一個很有教學價值的現象：同一段給號邏輯裡，三代擴充技術依序都會被呼叫，不是互斥的：

1. 新式 **BAdI** (**GET_BADI_g_badi_batch_number_int / CALL_BADI ...->init / ...->next_batch_number**) ——對應到 Enhancement Spot **ES_BATCH_NUMBER_INT**、BAdI 定義 **BADI_BATCH_NUMBER_INT** (套件 **VB**，**singleUse="true"**，Interface **IF_EX_BADI_BATCH_NUMBER_INT**)
2. **Classic User-Exit** (**CALL_CUSTOMER-FUNCTION '001' / '002'**) ——這就是本題要用的：
EXIT_SAPLV01Z_001 (Determination of Internal Number Ranges for Automatic Batch Number) 與 **EXIT_SAPLV01Z_002** (Modification of New Batch Numbers from Internal Assignment)，兩者都掛在 SAP Enhancement **SAPLV01Z** (說明：「CFCs for internal batch number assignment」，Function Group **XVBZ**，套件 **VBEX**) 底下
3. **S/4HANA Cloud BAdI** (**g_badi_batch_number_cust**，**IF_LOBM_BATCH_NUMBER**) ——最新一代的 Key User Extensibility 機制

三者的參數幾乎一模一樣 (**NR_RANGE_NR / OBJECT** 預設 '**BATCH_CLT**' / **SUBOBJECT** / **TOYEAR**)，這證實了 en01 提到的現象不是特例：**SAP** 在同一個擴充點上，把新舊技術疊在一起，不會因為有了新技術就砍掉舊的，舊系統用 Classic User-Exit 寫的客製化，升版後照樣有效。

Internal vs External：先搞懂這是給號「政策」，不是技術二選一

Classic User-Exit **SAPLV01Z** (本題用的) 只在內部給號 (**Internal**) 這條路徑上被呼叫；還有一個平行的 Enhancement **SAPLV1ZE** (說明：「CFCs for external batch number assignment」) 專門服務外部給號 (**External**) 路徑。這兩者是 Customizing 設定 (Batch Level / 批號指派方式，依物料主檔/工廠層級決定，對應到程式裡看到的 **T156-CHNEU** 欄位)，同一個系統裡不同物料可以分別設定，不是全系統只能選一種：

- **Internal (本題情境)**：系統自己用 Number Range Object 產生批號，使用者不用輸入。適用於批號純粹是內部技術追蹤用途、沒有必要對應外部編號的情境——想要統一格式、避免人工輸入錯誤 (我們的「日期+流水號」正是典型 Internal 場景)。

- **External**：批號由使用者手動輸入、或從外部系統帶進來，SAP 不自動產生。最常見的理由是收貨時要保留供應商自己的 **Lot Number**（尤其食品/藥廠等法規要求可追溯性的產業，SAP 批號要跟供應商包裝上的 Lot Number 一致，才能做到完整追溯）、或透過 IDoc/EDI 從外部系統帶入既有批號、或品管人員要目視核對標籤後手動輸入。

EXIT_SAPLV01Z_001 / _002 正規分工（本題兩支都已實作）：

- **EXIT_SAPLV01Z_001**（Include **ZXVBZU01**）：在取號之前執行，**CHANGING** 參數可以把 **OBJECT**（預設 'BATCH_CLT'）、**NR_RANGE_NR**（預設 '01'）換成自己的 Number Range Object——決定「跟哪個號碼簿要號碼」。用在你想換掉來源的時候（例如不同工廠要用不同的 Number Range Object）。本題把 **OBJECT** 重導向成自建的 **ZEN02BAT**。
- **EXIT_SAPLV01Z_002**（Include **ZXVBZU02**）：在取號之後執行，**CHANGING NEW_CHARG** 拿到剛取到的號碼（此時已經是從 **ZEN02BAT** 取出的 10 碼補零數字）、可以重新加工——決定「拿到號碼之後怎麼組成最終字串」。本題把它改成 **YYMMDD**+後4碼流水號。

兩支合起來的效果：**_001** 決定「用哪個號碼簿」，**_002** 決定「號碼長什麼樣子」——分工單純、各司其職，比一開始把兩件事都塞進 **_002** 自己重新呼叫 **NUMBER_GET_NEXT** 的權宜寫法更貼近 SAP 原始設計意圖。

啟用步驟（Classic User-Exit 要真的生效，一定要走 CMOD）：寫程式碼進 Include 並用 ADT 啟用，只代表原始碼存在系統裡，Function Group 實際載入執行的版本不會反映，直到走完整套 **CMOD** 流程：① 建 Enhancement Project（本題使用者建的是 **ZBATCHNO**）② Assign Enhancement **SAPLV01Z**（畫面上會列出這個 Enhancement 底下的所有 Component，含 **EXIT_SAPLV01Z_001/_002**）③ 雙擊每一個要用的 **Component** 進去編輯一次——這一步除了打開 Include 給你/改，第一次雙擊還會觸發系統自動生成該 **Include** 的骨架（本題 **ZXVBZU01** 就是這樣被生出來的，套件落在 **\$TMP**；**ZXVBZU02** 因為 2023 年就已經被人生成過，直接看到內容）④ **Activate Project**——這才是真正的開關。這整套是 GUI-only 步驟，**CMOD** 沒有 ADT API（見 [.claude/rules/sap-adt-mcp.md](#) 第 20 節），由使用者在 SAP GUI 操作，Claude 只能把 Include 內容準備好、驗證語法沒問題。✅ 本題已完整走完這四步並實測成功——使用者用真實採購單 **4500001919** 做 Goods Receipt（**MIGO**，動作 **101**），過帳 Log 顯示「Creating batch 2607290004」，批號格式（**YYMMDD**+4碼流水號）完全正確，證實 Enhancement 真的生效了。

Number Range Object（NROB）概念：一個「號碼簿」，本身有名稱（如 **BATCH_CLT**）、一或多個「Interval（區間）」（用二字节代碼區分，如 '01'），每個 Interval 記錄 From / To / 目前用到哪裡（**NRLEVEL**）。ABAP 呼叫 **NUMBER_GET_NEXT**（**EXPORTING nr_range_nr / object / IMPORTING number**）就能拿到「這個 Interval 目前的下一號」，系統自動處理並發（同時兩個人呼叫不會拿到同一號）。⚠️ 這個工具本身完全沒有 **ADT API**（跟 **SM59/DBCO/SPAD/Search Help** 同一類 GUI-only），只能在 **SNRO** 交易碼手動建立與維護 Interval；**NUMBER_GET_NEXT** 回傳的 **number** 一律是 10 碼、左邊補零的數字字串，不管 Interval 本身定義的號碼位數是多少（本題已用真實執行結果驗證：**0000000001 / 0000000002 / 0000000003**）。

⚠️ **重要實務教訓：Number Range 取號是「非交易性（non-transactional）」的，號碼一定會跳號，這是正常設計不是 bug**——實測發現：驗證程式 **ZR_EN02_BATCH_DEMO** 跑了 3 次（消耗 1/2/3），之後在 **MIGO** 真實過帳前，使用者的操作過程（例如按過 **Check** 按鈕、或重新輸入重試）又額外消耗了幾號，最終真正過帳成功寫進物料憑證的批號序號是 4（**2607290004**），但事後查 **NRIV** 的 **NRLEVEL** 已經來到 10——中間 5~9 這幾號被「用掉但沒有對應到任何真實批號」，永遠消失、不會被回收。原因是 SAP 的 Number Range 設計成非交易性：取號動作不受資料庫交易的 COMMIT/ROLLBACK 約束（效能考量，避免大量並行過帳互相鎖等待），所以只要程式邏輯跑到了取號那一行（即使是 **Check** 模擬、即使最後整筆交易被取消或失敗），號碼就真的被領走、永遠回不去。這代表批號/單號這類用 Number Range 產生的編號，天生就會有缺號，不能拿來當作「總共發生過幾筆交易」的計數依據，這是所有用過 Number Range Object 的人都要有的心理準備。

學習目標

- 能講出 Classic User-Exit (`EXIT_SAPLV01Z_001/_002`)、新式 BAdI (`BADI_BATCH_NUMBER_INT`)、Cloud BAdI 三代技術在同一個標準流程裡並存的現象，理解「新技術不會取代舊技術」
- 能講出 Internal / External 批號給號政策的差異與各自適用情境，知道這是 Customizing 層級的決策，不是程式技術上的二選一
- 能分辨 `_001` (取號前，決定用哪個 Number Range Object) 與 `_002` (取號後，加工最終字串) 兩個插入點的職責差異，並能各自寫出正確的程式碼
- 能講出 Number Range Object 的核心概念 (Object / Interval / NRLEVEL)，知道要用 `SNRO` 手動建立，`NUMBER_GET_NEXT` 回傳值一律 10 碼補零
- 能講出批號欄位 `CHAR(10)` 的長度限制如何反過來限制了「日期+時間+流水號」格式的設計取舍
- 知道 `ZX` 開頭的 Include 是保留給 Exit Function Group 的特殊命名空間，一般 ADT Include 建立 API 建不出來，但 CMOD 裡雙擊該 Component 可以觸發系統自動生成
- 能講出 Classic User-Exit 完整生效的四個步驟 (建 Project → Assign Enhancement → 編輯 Component → Activate Project)，理解「Include 原始碼存在」跟「Enhancement 生效」是兩件事

事前準備 (已於本系統 client 130 實際完成，非假設)

1. **Number Range Object ZEN02BAT**：使用者於 `SNRO` 手動建立，Number Length Domain 填 `CHARG` (查證自標準物件 `BATCH_CLT` 在設定表 `TNRO` 的 `DOMLEN` 欄位，同一個 Domain)，Interval `01`：`0000000001 ~ 9999999999`，未勾 External。已用資料預覽 API 查 `TNRO` / `NRIV` 兩張表確認設定正確。
2. **EXIT_SAPLV01Z_002 的 Include ZXVBZU02**：查證時發現這個 Include 已經是一個真實物件 (套件 `ZPP`，2023-08-08 由另一位同事 `MAVIS` 建立，但內容是空的)；進一步查 `MODSAP` / `MODACT` 兩張表確認 Enhancement `SAPLV01Z` 從未被任何 **CMOD Project** 正式指派，SE10 也查無掛著這支程式的傳輸請求——判斷是當年被雙擊觸發自動產生，但沒人接著建 Project 或寫程式碼的殘留物件。**確認同事無異議後**，用傳輸請求 `S4HK901982` (套件 `ZPP`) 寫入。
3. **CMOD Project ZBATCHNO**：使用者建立，Assign Enhancement `SAPLV01Z`。雙擊 `EXIT_SAPLV01Z_001` 觸發系統自動生成 Include `ZXVBZU01` (落在套件 `$TMP`，不需要傳輸請求)；`EXIT_SAPLV01Z_002` 因為 `ZXVBZU02` 早就存在，直接看到內容。
4. **ZXVBZU01 寫入重導向邏輯**：`OBJECT` 改成 `'ZEN02BAT'`；`ZXVBZU02 改寫成單純加工`：`new_charg = sy-datum+2(6) && new_charg+6(4).` (不再自己呼叫 `NUMBER_GET_NEXT`，因為 `_001` 已經把來源換成 `ZEN02BAT`，`_002` 拿到的 `NEW_CHARG` 已經是從 `ZEN02BAT` 取出的號碼)。兩支都已用 ADT 寫入並啟用 (`sap_inactive_objects` 確認無殘留未啟用版本)。
5. **驗證程式 ZR_EN02_BATCH_DEMO (\$TMP)**：不動任何真實貨物移動，單獨呼叫三次 `NUMBER_GET_NEXT` 驗證 `ZEN02BAT` + 格式化邏輯本身 (不經過 Enhancement 機制)，已用 `programrun` 無頭執行，真實輸出：

```
`` raw number: 0000000001 batch no: 2607280001 raw number: 0000000002 batch no: 2607280002 raw number: 0000000003 batch no: 2607280003 ``
```

(執行當下系統日期 2026-07-28，`sy-datum+2(6)` = 260728，補上 4 碼流水號 = 10 碼批號)

題目需求

1. 完成三代擴充技術對照表：技術世代 / 呼叫語法 / 對應物件名稱 / 是否需要 CMOD Project。
2. 解釋 `_001` / `_002` 的分工，並指出為什麼「兩支都用」比「只用 `_002` 自己重新呼叫 `NUMBER_GET_NEXT`」更貼近 **SAP 原始設計** (提示：對照 Lecture 的正規分工說明，想想如果之後要換

Number Range Object，只改 `_001` 一行 `OBJECT` 賦值，跟要重寫整段 `NUMBER_GET_NEXT` 呼叫，哪個維護成本更低）。

3. 講出 **Internal** 與 **External** 批號給號的差異與各自適用情境，並說明 `SAPLV01Z` 跟 `SAPLV1ZE` 兩個 Enhancement 的關係。
4. 讀懂 `ZXVBZU01` / `ZXVBZU02`（見答案快照），指出：`ZXVBZU01` 裡的 `OBJECT` / `NR_RANGE_NR` / `SUBOBJECT` 為什麼不用自己宣告（提示：它們是 `EXIT_SAPLV01Z_001` 的 `CHANGING` 參數，Include 是直接嵌進 FUNCTION 內部，參數在整個 Include 範圍內都可以直接當變數用）。
5. 情境判斷：如果之後想要「依工廠分開計數」（不同工廠各自一組流水號，互不影響），Number Range Object 要怎麼調整？該改 `_001` 還是 `_002`？（提示：見 Lecture 提到的 Subobject 概念，想想「換來源」跟「換格式」是哪一支的職責）

參考答案

三代擴充技術對照表：

技術世代	呼叫語法	對應物件	需要 CMOD Project？
Classic User-Exit	<code>CALL CUSTOMER-FUNCTION '001'/'002'</code>	<code>EXIT_SAPLV01Z_001/_002</code> （Enhancement <code>SAPLV01Z</code> ）	是，缺一不可——一定要用 <code>CMOD</code> 建 Enhancement Project、Assign <code>SAPLV01Z</code> 、Activate Project，這個 Enhancement 才會真的生效；Include 原始碼寫好、用 ADT 啟用過，只代表原始碼存在系統裡，Function Group 實際載入執行的版本不會反映，直到 Project 被 Activate（第一版教材誤判成「不需要 Project」，已更正）
Classic BAdI（新式，Enhancement Spot 管理）	<code>GET BADI / CALL BADI</code>	<code>BADI_BATCH_NUMBER_INT</code> （Enhancement Spot <code>ES_BATCH_NUMBER_INT</code> ）	否，直接 SE18/SE19 建 Implementation
S/4HANA Cloud BAdI	透過 <code>g_badi_batch_number_cust</code> （ <code>IF_LOBM_BATCH_NUMBER</code> ）	Key User Extensibility BAdI	否，走 ABAP Cloud 擴充性框架

`_001` / `_002` 分工與維護成本對比：`_001` 只做一件事——把 `OBJECT` 換成 `'ZEN02BAT'`，決定「跟哪個號碼簿要號碼」；`_002` 只做一件事——把拿到的號碼加上 `YYMMDD` 前綴，決定「號碼長什麼樣子」。這種切法之所以比「全部塞進 `_002` 自己重新呼叫 `NUMBER_GET_NEXT`」更好維護：以後如果要換 **Number Range Object**（例如改成依工廠分開計數），只要改 `_001` 一行 `OBJECT` / `SUBOBJECT` 賦值，`_002` 的格式化邏輯完全不用動；反過來，如果格式要改（例如流水號從4碼改5碼），只要改 `_002`，`_001` 完全不用動——兩支各自獨立變化，不會互相牽動。（最初卡在 `ZXVBZU01` 這個 Include 從未被生成過、且 `ZX` 開頭 Include 保留給 Exit Function

Group 專用、無法用標準 ADT Include 建立 API 生出來，必須在 CMOD 裡雙擊 EXIT_SAPLV01Z_001 才會觸發生成——這步驟完成後才補上了 _001。)

Internal vs External：SAPLV01Z (Internal) 與 SAPLV1ZE (External) 是兩個平行、互斥的 Enhancement，同一個物料在某個時間點只會走其中一條路徑，由 Customizing (Batch Level / 批號給號方式) 決定。Internal 適合批號只是內部技術追蹤用途、想要統一格式的情境；External 適合要保留供應商 Lot Number、或需要人工核對輸入的情境。兩者不是「哪個比較新/比較好」的技術升級關係，是不同業務需求對應不同政策。

情境判斷 (依工廠分開計數)：Number Range Object 要重新設計成帶 Subobject (在 SNRO 建立時填 Subobject Data Element，例如工廠代碼 WERKS_D)，每個工廠各自維護一組 Interval；這個改動屬於「換來源」，要改 _001——NR_RANGE_NR/OBJECT/SUBOBJECT (傳入實際工廠代碼) 都是 _001 的 CHANGING 參數，系統會依 OBJECT+SUBOBJECT 的組合各自計數，不會互相影響；_002 的格式化邏輯完全不需要修改。

思考題

- 三代技術裡，_001 / _002 這種 Classic User-Exit 沒有 BAdI 那種 TRY...CATCH cx_badi_not_implemented 的保護機制，這代表什麼實務風險？(提示：如果 Include 裡的程式碼寫錯、丟出未攔截的例外，會直接讓整個 VB_NEXT_BATCH_NUMBER 呼叫中斷，不像 BAdI 沒實作時會被優雅地跳過——Classic User-Exit 的程式碼品質要求其實更高，因為沒有安全網)
- ZXVBZU02 這個 Include 原本是套件 ZPP 的孤兒物件 (從未被任何 CMOD Project 指派過)，現在已經被 ZBATCHNO 這個 Project 正式認領——如果之後 PP 團隊 (MAVIS 原本可能想拿來做別的事) 也想用 EXIT_SAPLV01Z_002 做他們自己的事，會發生什麼衝突？(提示：一個 Function Exit 的 Include 只有一份，兩個團隊的需求會被迫寫在同一支程式碼裡，這正是為什麼動手前跟同事確認這一步在真實企業環境中不能省略——不像新建 Z 物件那樣互不干擾；現在已經被 ZBATCHNO 正式指派，後續若有人也想用同一個 Enhancement，會在 CMOD 看到已經有 Project 佔用，能及早發現衝突)
- 為什麼 NUMBER_GET_NEXT 的 number 輸出參數固定是 10 碼補零，而不是依 Number Range Object 定義的區間位數決定長度？這對本題「取後 4 碼當流水號」的寫法有什麼啟示？(提示：不管 Interval 定義多少位數，程式都要自己決定「取哪一段」，+6(4) 這種寫法要跟 Interval 實際的最大值位數對齊，如果 Interval 改成 5 位數上限就要跟著改成取後 5 碼，否則流水號可能被截斷或補零位置算錯)
- 本題實測發現 ZEN02BAT 的計數器很快就從 3 跳到 10，中間有 6 個號碼「消失」——如果這是一個有嚴格法規要求「單號必須連號、不可跳號」的情境 (例如某些國家的統一發票號碼)，Number Range Object 這種非交易性的取號機制還適用嗎？(提示：不適用，這類情境通常要用完全不同的機制——交易性、有鎖定機制的自訂邏輯，甚至需要人工事後稽核缺口，Number Range Object 是 SAP 為了效能刻意放棄「絕對連號保證」換來的設計，兩種需求在根本上衝突，選型前要先確認業務對「跳號」的容忍度)

答案

ZEN02BAT (Number Range Object，使用者於 SNRO 建立，Domain CHARG，Interval 01：0000000001 ~ 9999999999)、ZXVBZU01 (EXIT_SAPLV01Z_001 客戶 Include，套件 \$TMP，透過 CMOD Project ZBATCHNO 雙擊自動生成，快照 zxvbzu01.prog.abap，內容為重導向 OBJECT = 'ZEN02BAT')、ZXVBZU02 (EXIT_SAPLV01Z_002 客戶 Include，套件 ZPP，傳輸請求 S4HK901982，快照 zxvbzu02.prog.abap，內容改為單純加工 NEW_CHARG)、ZR_EN02_BATCH_DEMO (驗證程式，\$TMP，快照 zr_en02_batch_demo.prog.abap) 均已建立並啟用 (sap_inactive_objects 確認無殘留未啟用版本)。已用 programrun 無頭執行驗證程式 (不經過 Enhancement 機制，直接呼叫 NUMBER_GET_NEXT)，真實輸出批號 2607280001 / 2607280002 / 2607280003，證實 ZEN02BAT + 格式化邏輯本身正確。三代擴充技術對照表、Internal/External 差異、_001/_002 正規分工、情境判斷見本題內文。

Classic User-Exit 啟用進度：✅ 端對端驗證成功。使用者建立 CMOD Project **ZBATCHNO**、Assign Enhancement **SAPLV01Z**、雙擊 **EXIT_SAPLV01Z_001** 觸發生成 **ZXVBZU01**、Activate Project——四個必要步驟全部完成。用真實採購單 **4500001919** 做 Goods Receipt (**MIGO** · 動作 **101**) 實測，過帳 Log 顯示「Creating batch 2607290004」，批號格式 (**YYMMDD**+4碼流水號) 完全正確，證實 **Enhancement** 對真實貨物移動確實生效。過程中意外發現一個重要的 Number Range 實務教訓：計數器從測試消耗的 **3** 跳到過帳後的 **10**，證實 SAP Number Range 取號是非交易性的——**Check** 等模擬動作也會真的消耗號碼、不會歸還，號碼有缺口是正常現象，不代表程式有問題 (詳見 Lecture)。